Bluestonex
The SAP UX experts

HRX

Design & Technical Document

The HRX app and the HRX service: pages, design system, architecture, services, data, security and deployment

VERSION	1.0
DATE	20 October 2026
STATUS	For review
PREPARED BY	Bluestonex — the SAP UX experts
PRODUCT	HRX — HR Suite (Drop 2)

Document control

ITEM	VALUE
Document	HRX — Design & Technical Document, version 1.0, 20 October 2026
App	github.com/davida012/HRXDROP2, branch wip/design-15-10-26-1to1, commit 395ffca SAPUI5 app in hrx2026/ (namespace bsx.hrx.hrx2026), MTA HRX2026
Service	HRXService (bluestonex-labs/HRXService, fork davida012/HRXService), branch dev, commit 104775e CAP project bsx-hrx, MTA bsx-hrx
Design reference	"HRX Drop 2 - 15.10.26.html" (docs/handoff-design-15-10-26/design)
Diagram	docs/design-tech/hrx-architecture.drawio (and .png)
How facts were checked	Every name, field, route and setting below was read from the code and configuration at the commits above, and the app was run against a local copy of the service with test data.

Typography. This document is set in Raleway, the app's own typeface. The font is embedded in the .docx; if a copy shows another font, install Raleway (Google Fonts, SIL Open Font Licence).

Contents

1 Introduction **4**
1.1 Scope 4
2 Product overview **5**
3 Users and roles **6**
4 Pages **7**
4.1 Overview (everyone) 7
4.2 Admin only (managers) 7
4.3 Dialogs and pop-overs 8
5 UI design system **9**
5.1 Typography 9
5.2 Colour 9
5.3 Layout and components 10
5.4 The shared modal 10
6 Architecture **11**
6.1 How a request travels 11
6.2 Inside the app 11
7 Services **13**
7.1 The HRX service (CAP) 13
7.2 The legacy adapters 14
7.3 SAP Document Management 15
7.4 Destinations 15
7.5 XSUAA 16
7.6 SAP HANA Cloud (HDI) 16
7.7 Email 16
8 Data model **17**
9 Security and roles **18**
10 Deployment **19**
10.1 The service MTA (bsx-hrx) 19
10.2 The app MTA (HRX2026) 19
10.3 Build and deploy 19
11 Local development and testing **21**
12 Known issues **22**
13 Risks **23**
Appendix A — Glossary **24**

1 Introduction

HRX is Bluestonex's HR suite: one SAPUI5 app in which employees book their time and leave and read the company's policies, and managers approve leave, follow up sickness, publish documents, run timesheet and utilisation reports and maintain resources, clients and projects. Drop 2 brings the whole app onto the 15.10.26 design and onto the HRX service, a CAP service on SAP BTP.

This document describes the product as built: who uses it and how, every page, the design system, the architecture, every service the app calls, the data model, security, deployment, local testing, and the known issues and risks. It is written for the developers and administrators who build, run and support HRX.

1.1 Scope

- **In scope:** the HRX2026 app (repository HRXDROP2) and the HRX service (repository HRXService, branch dev), and the SAP BTP services they use.
- **Out of scope:** the older XSA/HANA backend that the app used to call (/services, destination bsxorgappsservices), except where the app still depends on it (section 7.6); SAP Process Automation, which shares the Document Management instance.

2 Product overview

HRX runs in the SAP BTP launchpad as a single app with a left-hand rail. The rail's **Overview** section is the same for everyone; the **Admin only** section appears for managers. Every page has the same frame: a page title ("HR SUITE" above the page name), an optional page action at the top right for managers, and on Home a health strip summarising the signed-in user's week.

AREA	WHAT HRX DOES
Time	Weekly timesheets per project with a comment for every booked day; Quick Timesheet on Home; timesheet compliance, reporting, utilisation and missing-timesheet views for managers; reminders by email.
Leave	Requests by day or half day (AM/PM), an approver, balance and quota; approve, reject and cancel; a team calendar with bank holidays; booking leave on someone's behalf.
Sickness	Absences recorded by managers, a policy trigger at three or more separate absences in a financial year (1 April – 31 March), follow-up messages, dismissals and a return-to-work checklist.
Documents	Policies and other documents published to the whole organisation (files in SAP Document Management), acknowledgement tracking and reminders.
Master data	Resources (employees and contractors, targets, working week, pictures), clients (contacts, support users, service levels, licensed components, application services, logos) and projects (resourcing, billing, areas, client contacts, attachments).
People	Everybody's picture and every client's logo wherever they appear, with initials as the fallback.

3 Users and roles

Anyone who can sign in to the subaccount and has a record in the HRX service's Users table (matched on their email) is an **employee**. The app derives the other roles; the service itself has no roles (section 9).

ROLE	WHO	WHAT THEY SEE	WHERE IT IS DECIDED
Employee	Every signed-in user held in Users	Overview pages: Home (own week), My Timesheet, My Leave, Team Calendar (own team's leave, no approvals), Policies to read, App Explorer	model/CurrentUser.js (profile from the service)
Manager	Anyone with at least one direct report (Users.Manager_EmployeeID), plus a short list of named people in MANAGER_EXCEPTIONS	Everything above, plus the manager dashboard on Home, approvals, page actions and the Admin only pages (Sickness, Documents, Timesheet reporting, Utilisation, Manage Resources, Manage Clients, Manage Projects)	CurrentUser.js isManagerException; HrxLegacy.js IsManager
Full access	A short list of named people in TIMESHEET_ALL_ACCESS (Timesheet.controller.js)	Every employee's timesheet in My Timesheet (merged from the team repo, PR #11)	Timesheet.controller.js _seesEveryone
View as (demo)	Testing aid: a picker under the signed-in user switches between Actual, Admin and Employee	Admin shows every manager page and everyone; Employee hides them	CurrentUser.js demoRole — to be removed before go-live

These roles only shape the UI. The service checks that a caller is signed in, not what they may do: an employee who calls the service directly could do anything a manager can. See risk R3 in section 13.

4 Pages

Routes are from manifest.json; every page is an XML view with its controller, fragments for its dialogs and its own stylesheet in css/pages/.

4.1 Overview (everyone)

PAGE (ROUTE)	PURPOSE	MAIN ACTIONS
Home (empty route)	The signed-in user's week at a glance. Managers also get the team dashboard.	Health strip pills (hours still to book, approvals pending, leave balance) open My Timesheet, Team Calendar and My Leave. Quick Leave (dates, type, full day/AM/PM, comments, approver). Quick Timesheet (project, hours, comment). This week's rings per working day (complete, short, missing, on leave, not due). Managers: Team in/working today, Timesheet compliance, Policy triggers, Documents awaiting acknowledgement, Team today, Pending approvals (approve/reject), Billable utilisation.
My Timesheet (timesheet)	Book the week's hours per project.	Week navigation, Add project(s), Delete project(s), hours per day with a required comment, Save. Leave and bank holidays show in their own rows and lock a day only when they cover all of it (a half day leaves the other half bookable). Tiles: target billable days, actual billed days, utilisation.
My Leave (leave)	Request leave and follow it.	Tiles: annual quota, days remaining, days booked. New leave request with a date-range picker, type, duration and approver; calendar of the month; requests filtered All / Requested / Approved / Rejected; edit or cancel a request still waiting.
Team Calendar (teamcalendar)	Who is away when.	Week grid with leave pills (pending ones hatched) and bank holidays; filters by team (My team / Everyone), resource and site. Managers also get Team leave: pending, approved and rejected requests over a chosen period, with Approve, Reject (with a note) and Cancel leave. The project filter is marked Not in MVP.
Policies to read (policies)	Read and acknowledge what the organisation publishes.	Open a document (from SAP Document Management), Acknowledge.
App Explorer (explorer)	Tiles for the HRX apps (model/apps.json).	Open a page. My Bonus and Manage Bonus have no service yet and say so.
Expenses	Placeholder in the rail ("Soon") for a separate app still to come.	—

4.2 Admin only (managers)

PAGE (ROUTE)	PURPOSE	MAIN ACTIONS
Sickness (sickness)	Follow up sickness under the absence policy.	Record sickness (page action: employee, dates, reason, notes, fit note). Policy triggers this financial year with absence history, Send follow-up message, Dismiss (with a note). Return-to-work cards with their checklist (Mark done, Mark RTW complete). All sickness this financial year.
Documents (documents)	Publish and track documents.	Upload document (page action: name, category, acknowledgement required, file). Acknowledgement progress per document, Remind outstanding (email), Unpublish.
Timesheet reporting (timesheetreporting)	Weekly bookings across the organisation.	Week, filters, totals per person and project, breakdown, compliance; Remind (email) the people under target.
Utilisation	Billable utilisation per	Period and filters (All, billable only, under 50 %, under 80 %).

PAGE (ROUTE)	PURPOSE	MAIN ACTIONS
(timesheetreporting?tab=util)	person.	
Manage Resources (manageresources)	Employees and contractors.	Search, filter by location, Add resource. Detail tabs: Info (with picture upload and Active switch), Manager, Working time (days, target hours, annual leave), Reward, Leave (their requests, Approve/Decline, Book leave on their behalf).
Manage Clients (manageclients)	Clients and what Bluestonex supports for them.	Search, Add client. Detail tabs: Basic info (location, logo, primary contact), Support team (client users), SLA (response and resolution hours per severity), Components (licensed software), Application services.
Manage Projects (manageprojects)	Projects, their resourcing and billing.	Search, filter, Add project. Detail tabs: Info (type, dates, PO, booking open, areas), Resourcing (assign, edit, release a resource with billing scheme, day rate, billable days, start date, areas), Contacts (the client's primary and secondary contact), Attachments.
Missing timesheets (missingtimesheets)	Who has not booked their time.	Reached from App Explorer and the Admin tiles.
Admin (admin)	A tile page of the admin apps (model/apps.json).	Open a page.

4.3 Dialogs and pop-overs

Every dialog uses the shared modal (section 5.4). The main ones: Quick Leave date picker (HomeDatePopover), Pending approvals and Decline (Home); Add project(s), Delete project(s), Comment and Week picker (My Timesheet); Edit leave, Half day, Cancel leave, Leave date picker (My Leave); Team leave request pop-over, Reject, Select resource(s) (Team Calendar); Record sickness, Follow-up message, Dismiss trigger (Sickness); Upload document, Unpublish (Documents); Add resource, Filter, Book leave, Decline leave (Manage Resources); Add client, Support user, SLA, Component, Application service, Confirm delete (Manage Clients); Add project, Assignment, Client contacts, Confirm (Manage Projects); the profile pop-over and notifications (rail).

5 UI design system

The app reproduces the 15.10.26 design on standard SAPUI5 1.150 controls (theme sap_horizon, compact). No control internals are replaced: css/style.css restyles them with the design's tokens, and each page's stylesheet (css/pages/*.css, loaded by manifest.json) is scoped under the page's root class.

5.1 Typography

FONT	USE	SOURCE
Raleway 400 / 500 / 600 / 700	All text: body 13.5 px, labels, tables, buttons, fields (token --sans)	Google Fonts, imported at the top of style.css
League Spartan 700 / 800	Display: page titles, card titles, KPI figures, modal titles (token --disp)	Bundled: fonts/league-spartan-700.woff2, -800.woff2
Tabler icons (subset)	The design's line icons: rail, tiles, field icons (sap-icon://tabler/<name>)	fonts/tabler (MIT licence), registered in Component.js with IconPool.registerFont
SAP icons	Icons the design does not override	SAPUI5 icon font

5.2 Colour

The tokens are defined once on :root in style.css and point the Horizon theme's own variables (--sapBrandColor, --sapButton_Emphasized_Background, ...) at the same palette.

SWATCH	TOKEN	HEX	USED FOR
	--dark	#0A0A0B	The side rail
	--ink / --txt	#33343A	Text
	--txt-2	#6D6E72	Secondary text, labels
	--txt-3	#9B9CA1	Hints, placeholders
	--frost	#F3F5F7	Page background, table heads, modal foot
	--line	#E3E6EA	Borders and rules
	--surface	#FFFFFF	Cards and dialogs
	--ice	#26AAE2	Brand: primary buttons, links, selection, focus
	--ice-deep	#1888BC	Primary hover
	--mint	#1FAE72	Success: approved, complete
	--amber	#D97706	Warning: pending, action needed
	--signal	#E5484D	Error: rejected, missing, policy trigger
	--over	#7C5CD6	Overbooked days
	(gradient partner)	#756EE5	Avatar and accent gradients: --ice to #756EE5

The HRX gradient. The logo runs magenta (about #9B39B0) through blue (#0089E3) to cyan (#00B3FD). In the app the same family appears as the --ice to #756EE5 gradient on initials avatars and accents; this document's cover band runs #9B39B0 → #756EE5 → #26AAE2 → #00B3FD.

5.3 Layout and components

- **Spacing and shape:** one gutter for the whole product (--gutter 28 px); radius 16 px for cards and dialogs (--radius), 11 px for fields and buttons (--radius-s); one soft shadow (--shadow).
- **Shell:** a dark side rail with the HRX logo, the Overview and Admin only sections, and the signed-in user (picture or initials, notifications bell, View as (demo)); the page header with the title and the page action.
- **Health strip:** pills for hours still to book, approvals pending and leave balance; each opens its page on click, Enter or Space.
- **KPI tiles:** a coloured top edge, an icon chip, a large League Spartan figure and a caption.
- **Cards and tables:** white cards with a title bar; sap.m.Table restyled with frost heads, row rules on the cells and a centred empty state.
- **Status pills:** Requested (amber), Approved (mint), Rejected (red), plus Working, On leave, Booking open and others.
- **Avatars:** the person's pictures from the pictures model (section 7.3), or their initials on the --ice → #756EE5 gradient.
- **Segmented filters:** IconTabBar and SegmentedButton styled as the design's pill switches (All / Requested / Approved …).

5.4 The shared modal

Every dialog takes class="hrxModal" (460 px; hrxModalWide 560 px): a white head with the title in League Spartan, the body, and a frosted foot with the action first and Cancel after it. Buttons combine hrxModalBtn with hrxModalBtnPrimary, hrxModalBtnDanger or hrxModalBtnGhost; text uses hrxModalText and field labels hrxModalLabel. The page behind is darkened (rgba(10,10,11,.55)) and slightly blurred.

6 Architecture

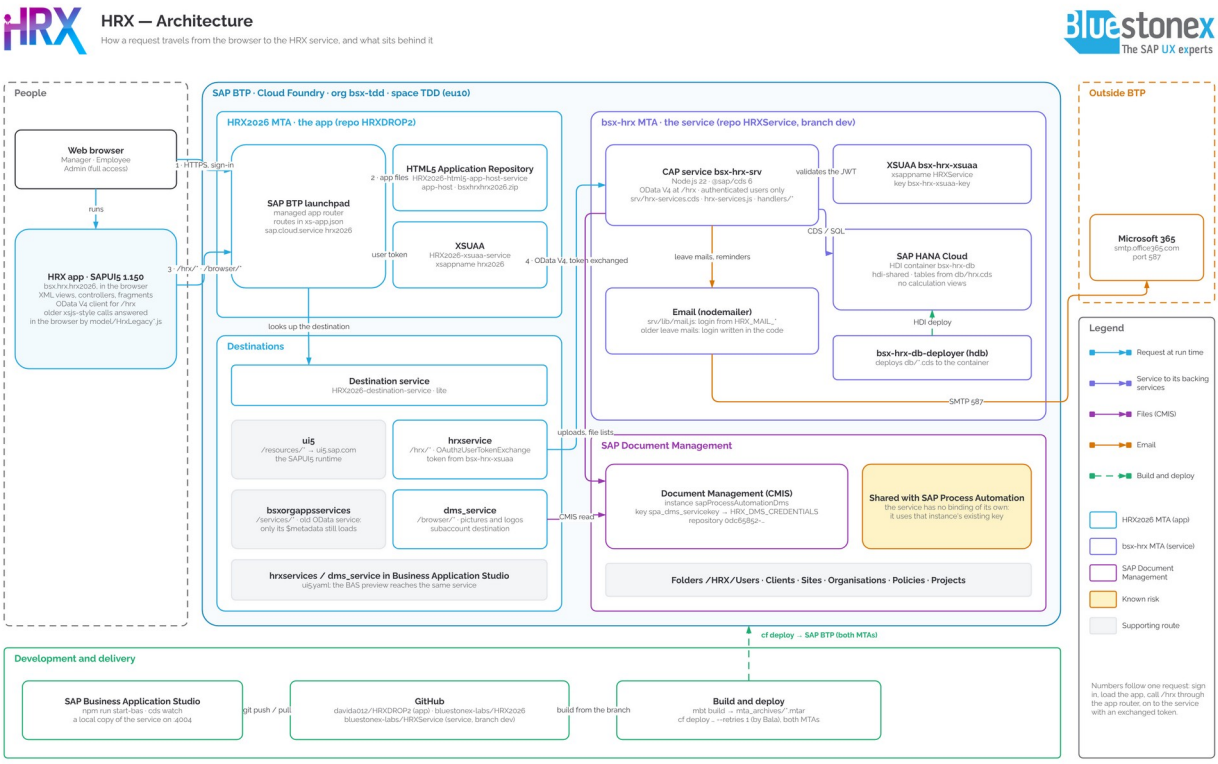


Figure 1 — HRX architecture (docs/design-tech/hrx-architecture.drawio).

6.1 How a request travels

- **1 · Sign-in.** The browser opens HRX in the SAP BTP launchpad. The managed app router signs the user in through XSUAA (HRX2026-xsuaa-service, xsappname hrx2026).
- **2 · The app.** The app router serves the app's files from the HTML5 Application Repository (HRX2026-html5-app-host-service, plan app-host); SAPUI5 itself comes from ui5.sap.com through the ui5 destination.
- **3 · Calls.** The app calls /hrx/* (OData V4) and /browser/* (pictures). xs-app.json routes them to the destinations hrxservice and dms_service.
- **4 · Service.** The hrxservice destination (OAuth2UserTokenExchange) exchanges the user's token for one issued by the service's own XSUAA instance bsx-hrx-xsuaa, so the CAP service knows who is calling. The service reads and writes its HANA HDI container, stores files in SAP Document Management and sends email through Microsoft 365.

6.2 Inside the app

LAYER	FILES	ROLE
Component	Component.js, manifest.json	Models (i18n, app, device, pictures, services), the Tabler icon font, the signed-in user (CurrentUser), the service's capabilities (HrxServices) and pictures (Pictures)
Shell	view/App.view.xml, controller/App.controller.js	Rail, page header, health strip, notifications, routing
Pages	view/*.view.xml, controller/*.controller.js, fragment/*.fragment.xml	One view and controller per page; dialogs as fragments

LAYER	FILES	ROLE
Service client	model/Hrx.js	OData V4 calls to /hrx: list, read, create, update, remove, actions, functions, uploads, picture URLs
Older calls	model/Backend.js, HrxLegacy.js, HrxLegacyClients.js, HrxLegacyProjects.js	The pages still speak the older xsjs/OData V2 shapes; these adapters answer them in the browser from /hrx (section 7.2)
Domain helpers	model/HrxTime.js, TeamLeave.js, SicknessPolicy.js, SicknessService.js, DocumentService.js, TimesheetReport*.js, NotificationService.js, Reminders.js, Mail.js	Calculations and service calls per area
Feature flags	model/HrxServices.js	Reads /hrx/$metadata once and switches features on when the service has them; otherwise the page shows a "Service needed" note

7 Services

7.1 The HRX service (CAP)

Project bsx-hrx, module bsx-hrx-srv: Node.js 22, @sap/cds 6, OData V4 service hrx at /hrx (srv/hrx-services.cds, @path: '/hrx', @requires: 'authenticated-user'). Handlers: srv/hrx-services.js and, for Drop 2, srv/handlers/ (clients, notifications, policies, projects, reminders, sickness). Every entity is a projection of db/hrx.cds (namespace bsx.hrx).

ENTITY SET	HOLDS	USED BY
Organisations, Sites	The organisation and its sites (with logo fields)	Every page (org, locations)
Users (+ WorkSchedule)	Employees: key EmployeeID (generated, BSX + 6 digits), names, WorkEmail, type, job title, site, manager, targets, picture (ImageRootID/ImageObjectID); working days, annual leave quota	Everywhere
LeaveType	Leave categories (Holiday, Sick, ...), accountable or not	Leave, Home, Team Calendar, Sickness
Leaves	One row per day: employee, category, NoOfDays, Start/EndDate, DayTime (Full Day/AM/PM), approver, Status_ID, comments, LeaveGrpID (one request)	Leave, Home, Team Calendar, Sickness, Resources
BankHolidays	Holidays per organisation and site	Timesheet, Leave, Team Calendar, Home
Clients, Contacts	Clients (logo fields); contacts with ObjectType 'C' for a client	Manage Clients, pictures
Projects, UserToProject, BillingScheme	Projects; assignments (key employee + project + billing scheme) with day rate, cost, billable days, dates; billing schemes	Manage Projects, Timesheet, reports
TimeLog	Hours booked per employee, project and day, with a comment	Timesheet, Home, reports
Policies, PolicyAcknowledgements	Published documents (file in Document Management) and who acknowledged them	Documents, Policies to read, Home
SicknessReviews, ReturnsToWork, ReturnToWorkSteps	Actions on policy triggers (EMAILED/DISMISSED); return-to-work processes and their checklist	Sickness, Home
ClientUsers, ClientSLAs, ClientComponents, ClientApplications	A client's support users, service levels, licensed components and application services	Manage Clients
ProjectAreas, ProjectAreaAssignments, ResourceAreaAssignments, ProjectClientTeam	Areas of work, the areas a project covers and each resource works in; client users on a project	Manage Projects (ProjectClientTeam not used by the app)
NotificationReads	Which app notifications a person has read	Notifications
Documents	Upload-only media entity: PUT Documents(objectID, objectType, objectFolderID)/content stores a file in Document Management for a User, Site, Org, Client, Policy or Project	Pictures, logos, policies, attachments
Bonus, BonusObjectives, Assets, AssetAssignment, ContactObjectTypes	Bonus and asset records	Not used by the app yet

ACTION / FUNCTION	WHAT IT DOES	CALLED BY THE APP
saveTimesheetEntry(timeLog), deleteTimesheetEntry(deleteEntry)	Insert or update / delete TimeLog rows	Yes
createLeaveRequest(userLog)	The signed-in user's request (status Requested); emails their manager	Yes
createLeaveRequestForTeamCalendar(userLog)	Leave booked on someone's behalf, approved at once (also sickness)	Yes
actionOnLeave(LeaveLog)	Approve (WFFlag true) or reject, with comments; emails the requester	Yes
sendReminder(Kind, EmployeeIDs, Subject, Message)	One email per person (srv/lib/mail.js); answers sent / failed	Yes
fetchReadNotifications(), markNotificationsRead(Keys)	Notification read state	Yes
acknowledgePolicy(PolicyID), removePolicy(PolicyID)	Acknowledge; unpublish (acknowledgements kept)	Yes
reviewSicknessTrigger, openReturnToWork, completeReturnToWorkStep, completeReturnToWork	Sickness follow-up	Yes
setProjectAreas, setResourceAreas	Replace a project's / a resource's areas	Yes
fetchProjectDocuments(ProjectID), deleteProjectDocument(ProjectID, ObjectID)	A project's attachments	Yes
fetchMyPolicies, fetchPolicyAcknowledgements	Policies with acknowledgement state	No (the app reads the entities)
fetchAssignments, fetchUserDetails, getUserDetail, fetchUserLeave, fetchLeaveAnalytics, leaveDates, leaveDatesForTeamCalendar, fetchTeamCalendar, leaveToApprove	Drop 1 functions	No (the app reads the entities)
fetchBonusHeader, fetchBonusList, fetchBonusUserList, submitBonus, fetchMyBonus	Bonus	No

Leave status IDs are fixed GUIDs that the app and the service agree on: Requested e7ab05aa-…, Approved e7ab02da-…, Rejected e7ab06f4-… (model/HrxTime.js, srv/hrx-services.js).

7.2 The legacy adapters

The pages were first built against an older XSA backend (xsjs endpoints and an OData V2 service at /services). Rather than rewrite every page, HrxLegacy.handle answers those calls in the browser from the HRX service. No legacy call reaches the old backend any more.

LEGACY CALL	ANSWERED BY	HRX SERVICE CALLS
timesheet/timesheet.xsjs (fetch, save, delete, fetchAssignments)	HrxLegacy.js Timesheet	UserToProject, TimeLog, Projects, Users; saveTimesheetEntry, deleteTimesheetEntry
hrx/leaveReqs.xsjs (fetchUser, getDates, requestLeave, update, delete)	HrxLegacy.js Leave	Users, LeaveType, Leaves, BankHolidays; createLeaveRequest / …ForTeamCalendar; PATCH/DELETE Leaves
hrx/leaveApprovals.xsjs (pendingApproval, action)	HrxLegacy.js Leave	Leaves; actionOnLeave
hrx/teamCalendar1.xsjs (team)	HrxLegacy.js Leave.team	Users, LeaveType, Leaves, BankHolidays

LEGACY CALL	ANSWERED BY	HRX SERVICE CALLS
master/manageUsers.xsjs (update, setActive, new)	HrxLegacy.js Users	Users, WorkSchedule, Documents (User)
hrx/manageClients.xsjs, supportx/ManageClientUsers.xsjs, supportx/ManageSLA.xsjs, hrx/manageClientApps.xsjs, hrx/manageClientServices.xsjs	HrxLegacy.js Clients, HrxLegacyClients.js	Clients, Contacts, ClientUsers, ClientSLAs, ClientComponents, ClientApplications, Documents (Client)
hrx/manageProjects.xsjs, hrx/manageAssignments.xsjs, hrx/manageAttachments.xsjs	HrxLegacy.js Projects / Assignments, HrxLegacyProjects.js	Projects, UserToProject, ProjectAreaAssignments, ResourceAreaAssignments; setProjectAreas, setResourceAreas, fetchProjectDocuments, deleteProjectDocument, Documents (Project)
OData V2 entity sets: Resources, LeaveTypes, sites, UserTypes, Clients, ProjectsData, PRIORITY, Softwares, Components, SubComponents, AppMaster, BTPServices, Task, Proj_Task_Assign_Text, PredefinedBilling	Backend.read → HrxLegacy.read	Users, LeaveType, Sites, Clients, Projects, ClientComponents, ClientApplications, ProjectAreas, BillingScheme (UserTypes, project types and priorities are fixed lists in the app)

7.3 SAP Document Management

ITEM	VALUE
Instance	sapProcessAutomationDms (the instance SAP Process Automation uses)
Service key	spa_dms_servicekey, given to the service as the environment variable HRX_DMS_CREDENTIALS (mta.yaml resource hrx-dms-key, type org.cloudfoundry.existing-service-key)
Repository	0dc65852-e10f-4a43-8cab-43397e9739e4 (repositoryId in srv/hrx-services.js)
Folders	/HRX/Users/<EmployeeID>, /HRX/Clients/<ID>, /HRX/Sites/<ID>, /HRX/Organisations/<ID> (file "profile.<ext>", replacing the previous one), /HRX/Policies/<ID>, /HRX/Projects/<ID>
Writes	The service, through the Documents entity: it creates missing folders (checking each level first), stores the file over CMIS and records the document ID on the record (ImageObjectID, LogoObjectID, FileObjectID)
Reads	The browser, through the app router's /browser/* route and the dms_service destination: /browser/<repository>/root?cmisselector=content&objectId=<ID> (model/Hrx.js pictureUrl)
Pictures model	model/Pictures.js reads every user's and client's picture IDs once (Users, Clients) into the pictures model: pictures>/users[EmployeeID], pictures>/clients[ClientID]; avatars fall back to initials. Reloaded after an upload.

7.4 Destinations

DESTINATION	DEFINED IN	ROUTE / USE	AUTHENTICATION
hrxservice	HRX2026 mta.yaml (instance-level, destination content module)	/hrx/* → https://bsx-tdd-tdd-bsx-hrx-srv.cfapps.eu10.hana.ondemand.com	OAuth2UserTokenExchange with bsx-hrx-xsuaa (key bsx-hrx-xsuaa-key)
dms_service	Subaccount (not in either MTA)	/browser/* → SAP Document Management (pictures, logos, files)	As configured in the subaccount
ui5	HRX2026 mta.yaml (destination service init data)	/resources/*, /test-resources/* → https://ui5.sap.com	None

DESTINATION	DEFINED IN	ROUTE / USE	AUTHENTICATION
bsxorgappsservices	Subaccount	/services/* → the older XSA backend; only the V2 model's $metadata is still loaded (manifest dataSource mainService)	None (xs-app.json)
hrx2026_HRX2026_html_repo_host, hrx2026_uaa_HRX2026	HRX2026 mta.yaml	HTML5 repository and the app's XSUAA for the launchpad	Service keys
hrxservices	Subaccount	Business Application Studio preview only (ui5.yaml)	As configured in the subaccount

7.5 XSUAA

- **App:** HRX2026-xsuaa-service (plan application, xs-security.json at the repository root: xsappname hrx2026, scope uaa.user, role template Token_Exchange). Signs users in to the app router.
- **Service:** bsx-hrx-xsuaa (plan application, HRXService xs-security.json: xsappname HRXService, scopes uaa.user and $XSAPPNAME.user, role template Token_Exchange; key bsx-hrx-xsuaa-key). CAP runs with auth: xsuaa in production; the app's hrxservice destination exchanges the user's token for one from this instance.

7.6 SAP HANA Cloud (HDI)

Resource bsx-hrx-db (com.sap.xs.hdi-container, service hana, plan hdi-shared). The module bsx-hrx-db-deployer (type hdb, path gen/db, @sap/hdi-deploy 5 with hdb 2 on Node 22) deploys the tables generated from db/hrx.cds by cds build --production (cds-dk 6). CAP connects with db.kind: hana-cloud. **No HANA calculation views are used**: every read is CDS over the tables.

The old backend. manifest.json still declares mainService (OData V2, services/services.xsodata/) and the default model preloads its metadata through bsxorgappsservices. Every read the pages make through it is answered from /hrx first (Backend.read → HrxLegacy.read); one write (Manage Clients application services) falls back to it only when the HRX service lacks ClientApplications, which dev has.

7.7 Email

- **Reminders** (sendReminder): srv/lib/mail.js with nodemailer, Microsoft 365 SMTP (smtp.office365.com:587 unless HRX_MAIL_HOST / HRX_MAIL_PORT say otherwise). The mailbox sign-in comes from HRX_MAIL_USER and HRX_MAIL_PASSWORD on the bsx-hrx-srv app; HRX_MAIL_DRY_RUN=true only logs. Without a mailbox the service answers "not set up" and the app opens a draft in the user's own mail app instead.
- **Leave emails** (new request to the manager; decision to the requester): older code in srv/hrx-services.js that builds its own nodemailer transport with a sign-in written in the code and ignores HRX_MAIL_DRY_RUN (risk R2).

8 Data model

The tables come from db/hrx.cds (namespace bsx.hrx). Most entities use CAP's cuid (UUID key ID) and managed (createdAt/By, modifiedAt/By) aspects; Users is keyed by EmployeeID. Associations become foreign-key columns named <association>_<key>, which is how the app writes them: EmpID_EmployeeID, LeaveCategoryId_ID, ApproverID_EmployeeID, Status_ID, Employee_EmployeeID, Project_ID, BillingID_ID, Client_ID, OrgID_ID, BaseSite_ID, Manager_EmployeeID, ClientID_ID.

ENTITY	KEY	MAIN RELATIONSHIPS
Organisations	ID	→ Sites (LinkToSite)
Users	EmployeeID	→ Organisations (OrgID), Sites (BaseSite), Users (Manager); WorkSchedule (composition); UserToProject, Leaves, Bonus, AssetAssignment
Clients	ID	→ Organisations; Projects (ClientID); ClientUsers, ClientSLAs, ClientComponents, ClientApplications; Contacts (ObjectType 'C', ObjectID)
Projects	ID	→ Organisations, Clients; UserToProject; ProjectAreaAssignments, ResourceAreaAssignments, ProjectClientTeam
UserToProject	Employee + Project + BillingID	→ Users, Projects, BillingScheme; TimeLog (by project and employee)
TimeLog	ID	→ Projects, Users; Date, Hours (Time), Comment
Leaves	ID	→ Users (EmpID, ApproverID), LeaveCategories, LeaveStatus; LeaveGrpID groups one request's days
Policies	ID	→ Organisations; PolicyAcknowledgements (→ Users)
ReturnsToWork	ID	→ Users; ReturnToWorkSteps (composition)
SicknessReviews	ID	→ Users; FiscalYear, Status, InstanceCount

Not exposed by the service: LeaveStatus (the app uses the fixed IDs in 7.1), UserTypes and ProjectTypes (the app has the lists: S Staff / C Contractor; TNM Time & materials, FXD Fixed price, INT Internal, FOC Free of charge).

9 Security and roles

- **Sign-in:** XSUAA through the managed app router (authenticationMethod route; every route except /services and the UI5 resources requires xsuaa). /hrx/* has CSRF protection.
- **Service access:** @requires: 'authenticated-user' on the whole service. There are no @restrict annotations and no handler checks a role, so every signed-in user can call every entity and action (risk R3).
- **Who is calling:** the service reads the user's email from the token (authInfo.getEmail()) and looks the employee up by WorkEmail. On dev that match is case-sensitive (issue I2).
- **Roles in the app:** derived from reporting lines and two named lists (section 3). The View as (demo) picker is a testing aid to be removed before go-live.
- **Secrets:** the Document Management key and the service's XSUAA key are bound as service keys; the mailbox for reminders comes from environment variables. The older leave emails carry a mailbox sign-in in the source (risk R2). HRXDROP2 is a public repository: keep credentials, .env files and internal-only material out of it.
- **Local testing never uses real credentials:** the test kit runs the service with mocked users, an in-memory database, a fake Document Management server and a stand-in for nodemailer (section 11).

10 Deployment

Both apps deploy to Cloud Foundry, org **bsx-tdd**, space **TDD** (eu10). The service URL in the app's destination, https://bsx-tdd-tdd-bsx-hrx-srv.cfapps.eu10.hana.ondemand.com, follows from that org and space.

10.1 The service MTA (bsx-hrx)

MODULE / RESOURCE	TYPE	NOTES
bsx-hrx-srv	nodejs (gen/srv)	CAP service; requires bsx-hrx-db, bsx-hrx-xsuaa, hrx-dms-key (as HRX_DMS_CREDENTIALS)
bsx-hrx-db-deployer	hdb (gen/db)	Deploys the tables to bsx-hrx-db
bsx-hrx-db	com.sap.xs.hdi-container	hana / hdi-shared
bsx-hrx-xsuaa	com.sap.xs.uaa	xsuaa / application, xs-security.json, key bsx-hrx-xsuaa-key
hrx-dms-key	org.cloudfoundry.existing-service-key	sapProcessAutomationDms / spa_dms_servicekey

Build: cds build --production with @sap/cds-dk 6 runs before all modules (mta.yaml build-parameters).

10.2 The app MTA (HRX2026)

MODULE / RESOURCE	TYPE	NOTES
bsxhrxhrx2026	html5 (hrx2026)	npm install; npm run build:cf (ui5 build preload with ui5-deploy.yaml, cachebuster)
HRX2026-app-content	com.sap.application.content	Uploads bsxhrxhrx2026.zip to the HTML5 repository
HRX2026-destination-content	com.sap.application.content	Instance destinations: html repo host, the app's XSUAA, hrxservice
HRX2026-destination-service	destination / lite	HTML5Runtime_enabled; the ui5 destination
HRX2026_html_repo_host	html5-apps-repo / app-host	HRX2026-html5-app-host-service
uaa_HRX2026	xsuaa / application	HRX2026-xsuaa-service, xs-security.json
bsx-hrx-xsuaa	org.cloudfoundry.existing-service	The service's XSUAA, for the token exchange

10.3 Build and deploy

Deploy the service first when both change (the app's destination needs bsx-hrx-xsuaa and the service URL). Build on a Mac with a Mac mbt (the one in node_modules is a Linux binary). Deployments are run by Bala.

Service (HRXService checkout, branch to deploy):
mbt build
cf deploy mta_archives/bsx-hrx_1.0.0.mtar --retries 1

App (HRXDROP2 checkout):
npm run build (rimraf resources mta_archives && mbt build --mtar archive)
cf deploy mta_archives/archive.mtar --retries 1

Before deploying, cf target -o bsx-tdd -s TDD. The dms_service, bsxorgappsservices and hrxservices destinations live in the subaccount and are not created by either MTA.

11 Local development and testing

- **Business Application Studio:** npm run start-bas / start-bas-flp at the repository root (ui5-bas.yaml); the preview reaches the service through the hrxservices destination and pictures through dms_service.
- **Local preview:** npx fiori run in hrx2026 with ui5.yaml sends /hrx to a local copy of the service on port 4004 (cds watch).
- **Test kit:** docs/handoff-design-15-10-26/tools. local-hrx-service/run-service.sh runs a copy of HRXService in memory with mocked @example.test users, a fake Document Management server (fake-dms.js, port 4005) and a stand-in nodemailer: it refuses to start unless every email is blocked. seed-for-app.js, seed-timesheets.js and seed-rich.js fill it with test records (pictures, leave in every state, sickness, policies, client and project details).
- **Checks:** one flow script per page (*-flows.js, puppeteer) that drives the page and checks the service's records; app.js, design.js and compare.py to compare the app with the design; detail-shots.js for every detail tab.
- **Linting:** the UI5 linter (npx @ui5/linter in hrx2026, or the ui5 MCP run_ui5_linter) on changed files.

12 Known issues

Found by checking every call the app makes against HRXService dev. Issues with a fix waiting are marked with its branch.

#	ISSUE	EFFECT	STATUS
I1	createLeaveRequest emails the requester's manager and throws when there is none	Anyone without a manager (a director) cannot request leave: 500, nothing saved	Fixed on HRXService branch fix/leave-requests-email-case
I2	The service matches the signed-in email to Users.WorkEmail case-sensitively	Someone stored as First.Last@... can open pages but every write fails (403 / "Employee doesn't exist")	Fixed on fix/leave-requests-email-case
I3	actionOnLeave emails only the first request's requester	Approving several people at once leaves the others without an email (the decisions save)	Fixed on fix/leave-requests-email-case
I4	The service requires a client support user's last name; the app (and design) do not	A support user with only a first name cannot be added (the error says why)	Fixed on fix/leave-requests-email-case
I5	Projects have no client contact fields	Manage Projects' Contacts tab shows "Service needed" and cannot save	Fields added on HRXService branch feature/project-client-contacts (the app switches the tab on by itself)
I6	Project attachments' "uploaded by" is Document Management's technical user	The uploader's name is not shown correctly	Open
I7	Client and project codes are the service's UUIDs	Lists show a long ID where the design shows a short code	Open (no short code in the service)
I8	My Bonus and Manage Bonus have no service in the app	App Explorer marks them as service needed	Open

13 Risks

#	RISK	WHY IT MATTERS	SUGGESTED ACTION
R1	Document Management uploads depend on a key shared with SAP Process Automation (sapProcessAutomationDms / spa_dms_servicekey)	That instance accepts no new bindings, so the service has no binding of its own. If the key is rotated or deleted for Process Automation, HRX uploads and pictures stop; the two products also share one repository's quota and access	Agree ownership of the key; plan a dedicated Document Management instance and repository for HRX
R2	The older leave emails use an SMTP sign-in written in the code (srv/hrx-services.js) and ignore HRX_MAIL_DRY_RUN	A credential in source control; a test run of that code sends real email	Move the leave emails onto srv/lib/mail.js and its environment variables; change the mailbox password
R3	No authorisation in the service beyond authenticated-user	Any employee calling the service directly can approve leave, edit resources, clients and projects, publish or remove policies and book time for others	Add roles to xs-security.json and @restrict / handler checks for manager actions
R4	Testing aids in the app: View as (demo) and named email lists in model/CurrentUser.js and Timesheet.controller.js	Anyone can switch to Admin view; access depends on code changes	Remove the picker before go-live; move the lists to the service
R5	The app still preloads the old OData V2 service's metadata (bsxorgappsservices)	An extra dependency on the old backend at start-up	Remove mainService from manifest.json once nothing needs it

Appendix A — Glossary

TERM	MEANING
CAP	SAP Cloud Application Programming Model; the HRX service is a CAP Node.js project
CMIS	The standard protocol SAP Document Management speaks (browser binding at /browser)
HDI	HANA Deployment Infrastructure: the service's tables live in one HDI container
MTA	Multi-target application: one deployable archive (.mtar) per repository
XSUAA	The SAP BTP authorisation service that issues the users' tokens
Destination	A named connection in the subaccount the app router uses to reach a service
Financial year	1 April – 31 March, as the sickness policy counts it